# Junior Java Backend fejlesztő I.

Java alapok I.

(adattípusok, operátorok, változók, vezérlőszerkezetek)

**Botlik Béla**

# A Java programnyelv I.

- A Java napjaink egyik legnépszerűbb objektumorientált programozási nyelve.
  - https://www.tiobe.com/tiobe-index → programnyelvek népszerűségi indexe.
- Története:
  - Előzmények: szoftverkrízis → objektumorientált programozás (OOP).
  - 1995-ben jött létre, James Gosling tervei alapján (Sun Microsystems).
  - 2009-től Oracle termék, amikor is az Oracle felvásárolta a Sun-t.

# A Java programnyelv II.

- Jellemzői:
  - Objektumorientált
    - A hierarchia az elsődleges, nem a műveletek.
  - Platformfüggetlen
    - Forráskód → bájtkód → Java virtuális gép (JVM).
  - Hálózati programozásra alkalmas
    - Kliens-szerver megvalósítások.
  - Távoli helyen történő biztonságos futtatás

# A Java programnyelv III.

- További jellemzői:
  - Többnyelvűség
    - Lokalizáció
  - Dinamikus továbbfejleszthetőség
    - Modularizáció
  - Skálázhatóság
  - C alapú
    - Szintaxisát főként a C nyelvtől örökölte.
    - Nevén kívül nincs szoros „rokonságban” a JavaScript nyelvvel.

# A Java programnyelv IV.

- Felhasználási területei:
  - vállalati szektor
  - pénzügyi szektor
  - e-kereskedelem
  - beágyazott rendszerek
  - oktatás

  de leginkább bárhol ☺

# Java szintaktika

- A formai szabályok be nem tartása esetén szintaktikai hiba miatt nem fordul le a program.
- A C nyelvből átvett legfontosabb elemek:
  - Minden parancsot pontosvesszővel zárunk.
  - A blokkokat kapcsos zárójelekkel határoljuk.
  - A program belépési pontja a main metódus.
  - Kommentezés
    - // egysoros megjegyzés
    - /* többsoros
      komment */

# Java adattípusok I.

- A Java erősen típusos nyelv
  - A típusnak fordításkor ismertnek kell lenni, ezért a változókat típussal kell létrehozni.
- Típus szerint megkülönböztetünk egyszerű (primitív) és összetett típust.
  - A primitív adattípusok érték szerint kerülnek tárolásra, míg az összetett adattípusok referencia szerinti tárolásúak.

# Java adattípusok II.

- Egyszerű (primitív) adattípusok:
  - Logikai érték tárolása:

| Megnevezés | Tárhely | Érték |
|---|---|---|
| boolean | 1 byte | false; true |

  - Karakter tárolása:

| Megnevezés | Tárhely | Érték |
|---|---|---|
| char | 2 byte | Unicode karakterkészlet |

# Java adattípusok III.

- Egész számok tárolása:

| Megnevezés | Tárolás | Érték |
|---|---|---|
| byte | 1 byte | -128 … 127 |
| short | 2 byte | -32768 … 32767 |
| int | 4 byte | -2147483648 … 2147483647 |
| long | 8 byte | -9223372036854775808 … 9223372036854775807 |

- Valós számok tárolása:

| Megnevezés | Tárolás | Érték |
|---|---|---|
| float | 4 byte | ±3.4028235E38 |
| double | 8 byte | ±1.79769313486231570E308 |

# Java adattípusok IV.

- Az összetett adattípusok egyszerű adattípusokból felépülő adatszerkezetek, melyek az adattárolás módját is meghatározzák.
  - Szöveg
    - String – immutable (változtathatatlan) objektum
    - StringBuffer, StringBuilder – változtatható tartalmú objektum
  - Tömb
  - Osztály
  - Interface

# Típuskonverzió I.

- Szigorúan típusos nyelveknél az értékadások mindkét oldalán azonos típusú adatoknak kell szerepelniük. Nem egyforma típusok esetén konverzióra van szükség. Konverziós lehetőségek:
  - Implicit konverzió
  - Explicit konverzió
- Ha sem implicit, sem explicit konverzió nem lehetséges, még létezhet olyan beépített metódus, ami elvégzi az átalakítást.
  - Pl: Integer.parseInt()

# Típuskonverzió II.

- Implicit konverzió → a fordító automatikusan hajtja végre, ha garantált, hogy nem lép fel adatvesztés.
  - Olyan típusokon hajtja végre, ahol a konvertálandó típus tárolási igénye kisebb, mint annak a típusnak a tárolási igénye amibe a konvertálást végezni kell. Bővítő konvertálásnak is hívják.
- Explicit, vagy szűkítő konverzió → a fordító nem végzi el automatikusan, azt nekünk kell a kódban kezdeményeznünk. Típusba kényszerítésnek vagy kasztolásnak is hívják:
  - Megvalósítása: (<céltípus>)<konvertálandó típus>
  - Könnyen adatvesztést eredményezhet, mert egy szűkebb értéktartománnyal rendelkező típusba konvertálunk.

# Java operátorok I.

- A különböző műveleti típusokat operátoroknak hívjuk, melyek a műveleteket operandusokon hajtják végre:
  - eredmeny = a – b;
    - ahol a és b operandusok, '-' az operátor, a teljes sor pedig egy értékadó utasítás, ami a két szám különbségét „eredmeny" változóba tölti be.
- Az operátorok a műveletek jellege szerint csoportosíthatók:
  - aritmetikai operátorok
  - relációs operátorok
  - logikai operátorok
  - értékadó operátorok
  - egyéb operátorok

# Java operátorok II.

| Műveleti típus | Operátorok |
|---|---|
| aritmetikai | + - * / % |
| relációs | < > <= >= == != |
| logikai | && \|\| ! & \| ^ ~ |
| értékadó | = += -= *= /= %= |
| egyéb | ?: [] . ++ -- << >> >>> |

# Java operátorok III.

- Összetett kifejezést tartalmazó utasítás műveleteit a precedencia szabály szerint kell kiértékelni.
  - Mindig a magasabb precedencia szinten álló műveletek hajtódnak előbb végre.
  - Azonos szinten álló műveletek balról jobbra haladva értékelődnek ki.
    - Kivétel az értékadás, ami jobbról balra megy végbe.
  - A kiértékelés sorrendjét zárójelezéssel befolyásolhatjuk.
- A teljes precedencia táblázat:
  - https://docs.oracle.com/javase/tutorial/java/nutsandbolts/operators.html

| Legfontosabb precedencia szintek: |
| --- |
| Unáris műveletek ( ++, --, ! ) |
| Multiplikatív műveletek ( *, /, % ) |
| Additív műveletek ( +, - ) |
| Relációs műveletek |
| Logikai műveletek |
| Értékadás |

# Változók használata I.

- A változó egy névvel ellátott -futás közben változtatható tartalommal bíró- átmeneti tároló.
- A változónak még az első használat előtt meg kell adni a típusát és az azonosítóját.
- A változó életciklusának elemei a deklaráció és a definíció.

# Változók használata II.

- Deklaráció: a változó bevezetése a programba a legelső használat előtt, típus és név megadásával.
  - Például:
    - int szam;
  - Azonosítóra vonatkozó névkonvenciók:
    - Az első karakter csak alfabetikus lehet.
    - Nem lehet azonos a programnyelv foglalt kulcsszavaival.

# Változók használata III.

- Definíció: a változó értékének beállítása.
  - Értéket csak deklarált változó kaphat.
  - Például:
    - szam = 5;
      - // az utasítást korábban megelőzte egy „int szam;” deklaráció
  - Deklaráció és definíció opcionálisan egyszerre is történhet:
    - int szam = 5;

# Hatókör és érvényességi kör

- A változó elérhetőségét és módosíthatóságát a deklaráció helye határolja be. Ez a változó hatóköre, működési területe:
  - blokkszintű
  - metódusszintű
  - osztályszintű
- Lokális – globális változók:
  - Lokális (helyi) változóknak nevezzük azokat a változókat, amelyek egy nagyobb egységből nem láthatók.
  - Globális változóknak a széles működési területű változókat hívjuk.
- Érvényességi kör – a változó időbelisége, létezése:
  - A változók az adott szint élettartama alatt használhatók.

# Konstans

- Konstans vagy állandó:
  - Olyan adattároló melynek értéke a program futása közben nem változtatható.
- Használata:
  - final <típus> <elnevezés> = <érték>
    - példa:
      - final double PI = 3.141592653589793

# Vezérlőszerkezetek I.

- Vezérlési szerkezetnek hívjuk az utasítások végrehajtásának sorrendjét meghatározó programelemet.
  - **szekvencia**: utasítások egymás utáni végrehajtása.
  - **elágazás**: döntési szerkezet, ami által a program ágakra bontható.
    - Egyszerű elágazás
    - Többágú elágazás
    - Esetszétválasztásos szerkezet
  - **iteráció**: ismételt utasításvégrehajtást kiváltó szerkezet.
    - Számlálós ciklus
    - Elöltesztelős ciklus
    - Hátultesztelős ciklus

# Vezérlőszerkezetek II.

- Egyszerű elágazás

```
if (feltétel) {
        utasítás(ok);
}
else {          // opcionális ág
    utasítás(ok);
}
```

Többágú elágazás

```
if (feltétel) {
        utasítás(ok);
}
elseif (feltétel) {
        utasítás(ok);
}
else {          // opcionális ág
        utasítás(ok);
}
```

# Vezérlőszerkezetek III.

- Esetszétválasztásos szerkezet példa:

```
switch (osztalyzat) {
case 1:
    System.out.println("elégtelen");
    break;
case 2:
    System.out.println("elégséges");
    break;
case 3:
    System.out.println("közepes");
    break;
case 4:
    System.out.println("jó");
    break;
case 5:
    System.out.println("jeles");
    break;
default:
    System.out.println(„hibás érték");
    break;
}
```

# Vezérlőszerkezetek IV.

- Számlálós ciklus:
  - for (inicializálás; feltétel; növekmény) {
    utasítás(ok);
    }
- Bővített számlálós ciklus:
  - for (<típus> elem : tároló) {
    utasítás(ok)
    }

# Vezérlőszerkezetek V.

- Elöltesztelős ciklus:
  - while (feltétel) {
    utasítás(ok);
    }
- Hátultesztelős ciklus:
  - do {
    utasítás(ok);
    } while (feltétel);

# Integrált fejlesztői eszköz (IDE)

- Hatékony és gyors szoftverfejlesztést lehetővé tevő eszköz
  - Grafikus felhasználói felület
  - Egy helyen elérhető fejlesztői eszközök
    - Szövegszerkesztő (editor)
    - Fordítóprogram/értelmező (compiler/interpreter)
    - Hibakereső (debugger)
    - Verziókezelő (version control)

# Telepítés I.

- https://www.oracle.com/java/technologies/downloads/#java21
  - Keresőben: jdk 21 download
  - Találati listában: Java SE 21 Archive Downloads – Oracle
  - Az oldalon az operációs rendszernek megfelelő letöltés kiválasztása
  - Oracle fiókba belépés/regisztráció
  - A letöltött telepítő indítása

# Java technológiai áttekintés és működés

- Platformok:
  - Java SE – Standard Edition: általános célú platform.
  - Java EE – Enterprise Edition: elosztott, hibatűrő rendszerek fejlesztéséhez.
  - Jakarta EE – JE-nek 2017-től az Eclipse Foundation általi fejlesztése.
- Futtató környezet:
  - JRE – Java Runtime Enviroment
- Fejlesztői környezet:
  - JDK – Java Development Kit
- JVM – Java Virtual Machine: virtuális gép
- Munkafolyamat forráskódtól futtatásig:
  - .java → javac → .class → JVM → platformfüggetlen futtatás

# Telepítés II.

- https://www.eclipse.org/downloads/packages/release/2024-06/r
  - Keresőben: eclipse 2024 06 r zip download
  - Találati listában: Eclipse IDE 2024-06 R Packages
  - Az oldalon az operációs rendszernek megfelelő letöltés (zip) kiválasztása
  - A letöltött zip fájl kicsomagolása
  - Első indítás előtt egy mappa készítése a leendő munkaterület részére
  - Első indításkor az elkészített mappa megadása munkaterületként

# Feladat I. – Hello World!

| Alacsony szintű programnyelvek | Magas szintű programnyelvek |
|---|---|
| | |
| **Assembly** | **Java** |
| section .text | public class HelloWorld { |
| global _start | public static void main(String[] args) { |
| _start: | System.out.println("Hello, World!"); |
| mov edx,len | } |
| mov ecx,msg | } |
| mov ebx,1 | |
| mov eax,4 | |
| int 0x80 | **Python** |
| mov eax,1 | print("Hello, World!") |
| int 0x80 | |
| section .data | |
| msg db 'Hello, world!',0xa | |
| len equ $ - msg | |
| | |
| **Gépi kód** | **C#** |
| ba66 000e 0000 b966 0024 | class Program { |
| 0000 bb66 0001 0000 b866 | static void Main(string[] args) { |
| 0004 0000 80cd b866 0001 | Console.WriteLine("Hello World"); |
| 0000 80cd 0000 6548 6c6c | } |
| 2c6f 7720 726f 646c 0a21 | } |

# Tiszta kód I.

- Programjainkat hatékonyabbá, átláthatóbbá, könnyebben értelmezhetővé tehetjük a „tiszta kód” alapelveinek alkalmazásával.
- Ajánlott irodalom:
  - Robert C. Martin: Tiszta kód (Clean Code) 2009.
    - Magyar kiadás: 2010 Kiskapu Kft. 467 oldal

# Tiszta kód II.

- A kód legyen -mások számára is- könnyen olvasható.
- Használjunk beszédes elnevezéseket változó, alprogram, paraméter, osztály esetén egyaránt.
  - Az elnevezés utaljon arra, hogy miért, mire, hogyan használatos.
    - osztály, objektum → főnév
    - alprogram → ige, igenév
  - Ne féljünk a hosszú nevek alkalmazásától.

# Adatbekérés I.

- A bemeneti (input) értékek egy része adatbekérés útján kerül a rendszerbe.
- Az adatbekérésre beépített osztályok állnak rendelkezésre:
  - Scanner
    - Scanner sc= new Scanner(System.in); // példányosítás
      - Az osztályt a java.util package tartalmazza, amit importálni szükséges.
    - String nev = sc.nextLine(); // adatbekérés
    - sc.close(); // lezárás → memóriafelszabadítás

# Adatbekérés II.

- BufferedReader
  - BufferedReader br = new BufferedReader(new InputStreamReader(System.in))
    - Az osztályt a java.io package tartalmazza, amit importálni szükséges.
  - String nev = br.readLine();
  - br.close();
  - Kivételkezelésre kötelez
  - Szinkronizált, többszálú és szálbiztos

# Adatkiírás

- Kiírás képernyőre:
  - System.out.print(<szöveg>);  // sortörés nélküli kiírás
  - System.out.println(<szöveg>);  // kiíratás sortöréssel
  - A szövegben elhelyezett vezérlőkarakterekkel határozható meg a megjelenítés módja:
    - https://www.w3schools.com/java/java_strings_specchars.asp
- Numerikus értékek formázott kiíratása
  - System.out.printf(<formázó karaktereket tartalmazó szöveg>, paraméterek);
  - System.out.format(<formázó karaktereket tartalmazó szöveg>, paraméterek);
  - A formázó karakterek listája:
    - https://docs.oracle.com/javase/tutorial/java/data/numberformat.html

# Feladat II. – adatbekérés, -kiíratás

- A., Kérje be a felhasználó vezetéknevét, majd új sorban a keresztnevét.
  - Írja ki a képernyőre egy sorban a felhasználó teljes nevét.
  - Használjon vezérlőkaraktereket a kiíratás során.

- B., Kérje be a konzolról egy téglalap két oldalának méretét.
  - Írja ki a képernyőre a téglalap kerületét és területét.
  - Formázza a kiírandó eredményt.